



Lab-02 Report

Course: CISC5950

Title: Big Data Computing

Submitted To:

Dr. Ying Mao

Associate Professor

Department of Computer and Information Science

Fordham University

Submitted By:

Muhammad Zawad Mahmud (ID# A23176102)

Samiha Islam (ID#A23171266)

Date of Submission: 03/30/2026

Dataset Overview

Part 1: NYC Parking Violations (Fiscal Year 2025)

We used the NYC Open Data set of parking tickets issued in NYC for Fiscal Year 2025 for Part 1. There are about 16.56 million records in the full dataset. Because our 3-node HDFS cluster doesn't have enough space, we only processed the first half of the dataset, which had **8,279,622** records.

There are 43 columns in each record of the dataset. These are: Summons Number, Plate ID, Registration State, Plate Type, Issue Date, Violation Code, Vehicle Body Type, Vehicle Make, Issuing Agency, Street Codes, Vehicle Expiration Date, Violation Location/Precinct, Issuer Precinct/Code/Command/Squad, Violation Time, Violation County, House Number, Street Name, Intersecting Street, Law Section, Sub Division, Days Parking In Effect, Vehicle Color, Vehicle Year, Meter Number, Feet From Curb, Violation Post Code, and Violation Description.

The raw data is in quoted CSV format, with dates in MM/DD/YYYY format and 12-hour time codes (for example, "0209P" for 2:09 PM). In the cleaned output, there were 44 columns because our cleaning pipeline standardized these formats, normalized color and state abbreviations, and added an extracted hour-of-day column.

Part 2: E-Commerce Clickstream Data

We used the **large** synthetic e-commerce clickstream dataset with about **2 million** user interaction events for Part 2. The dataset runs from December 4, 2025 to May 2, 2026 and has 10 columns: event_id, user_id, timestamp, session_id, event_type (view, addtocart, purchase, removeitem), product_id, category, price, device_type (desktop, mobile, tablet), and traffic_source (direct, search, social, email, ads). Our session reconstruction pipeline found 19,448 different users in the dataset.

Part 1: Urban Analytics Pipeline (NYC Parking Data)

Task 1.1: Data Quality Assessment and Cleaning

This task sets up a two-job MapReduce pipeline. The cleaning mapper reads each row of the CSV file and does five things: it checks that plate_id, violation_code, and issue_date are not empty; it checks that the date is between 2024 and 2025 (it works with both ISO and MM/DD/YYYY formats); it uses a lookup dictionary with more than 60 entries to standardize vehicle colors; it uses a lookup dictionary with more than 60 entries to standardize state abbreviations; and it gets the hour of day (0-23) from the violation_time field and adds it as a 44th column. One of the biggest problems was dealing with Hadoop's multi-mapper file splits. Only the first of 31 mappers gets the header, so the mapper looks for known column names and uses positional indices when the header isn't there. The reducer is an identity pass-through that removes extra spaces at the end. The quality mapper uses the same logic to check for errors, but it sends out counter names (like TOTAL_INPUT, VALID_OUTPUT_RECORDS, etc.) instead of cleaned rows. The quality reducer adds these counters together and figures out the percentages.

Mapper/Reducer I/O

Job 1 (Clean):

Component	Format
Mapper Input	Raw CSV row (43 columns, quoted fields)
Mapper Output	Cleaned CSV row with normalized fields + appended hour_of_day (44 columns)
Reducer Input	Pass-through (identity reducer)
Reducer Output	Cleaned CSV row written to /parking/clean/

Job 2 (Quality Report):

Component	Format
Mapper Input	Raw CSV row (43 columns)
Mapper Output	counter_name \t 1 (e.g., VALID_OUTPUT_RECORDS \t 1)
Reducer Input	Sorted counter_name \t count pairs
Reducer Output	counter_name \t total_count \t (percentage%)

Data Quality Report

Metric	Count	Percentage
Total input records	8,279,622	-
Valid output records	8,278,399	100.0% (rounded up)
Invalid date records	525	0.006%
Missing critical fields	698	0.008%
Color standardizations	6,181,039	74.7%
State code corrections	0	0.0%

```
root@node-0:/mapreduce-test/mapreduce-test-python/lab2/task1_1# cat task1_1_quality_report.csv
TOTAL_INPUT      8279622
VALID_OUTPUT_RECORDS  8278399 (100.0%)
INVALID_DATE_RECORDS  525      (0.0%)
MISSING_CRITICAL_FIELDS  698      (0.0%)
COLOR_STANDARDIZATIONS  6181039 (74.7%)
STATE_CODE_CORRECTIONS  0         (0.0%)
```

Figure: Data Quality Report

Task 1.2: Peak Enforcement Time Analysis

This task uses a pipeline with two jobs. The hourly mapper (`mapper_hourly.py`) reads each cleaned CSV row and gets the `hour_of_day` from the last column (index -1). It checks that the hour is a whole number between 0 and 23, then sends the hour as the key and 1 as the value. The standard is used by the hourly reducer (`reducer_hourly.py`). The MapReduce word-count aggregation pattern keeps a running sum for the current key. When a new key comes in (Hadoop guarantees sorted input), it outputs the total for the previous key and resets the counter.

The window mapper (`mapper_window.py`) takes the hourly output (pairs of hours and counts) and sends out every record again with the key "ALL." This makes all 24 hours go into one reducer. The window reducer (`reducer_window.py`) puts all 24 hourly counts into a dictionary and then checks every possible 4-hour window by moving the window. It goes through the start hours from 0 to 23 and adds up the counts for four hours in a row, going back to 0 at midnight. The window with the most tickets is chosen and its start hour, end hour, and total ticket count are shown.

Mapper/Reducer I/O

Job 1 (Hourly Counts):

Component	Format
Mapper Input	Cleaned CSV row (reads hour_of_day from last column)
Mapper Output	hour \t 1
Reducer Input	Sorted hour \t count pairs
Reducer Output	hour \t total_tickets

Job 2 (Peak Window):

Component	Format
Mapper Input	hour \t total_tickets (from Job 1 output)
Mapper Output	ALL \t hour \t count (single key to force one reducer)
Reducer Input	ALL \t hour \t count (all 24 hours in one reducer)
Reducer Output	Start_Hour \t End_Hour \t Total_Tickets

Hourly Distribution

Hour\Tickets	
0	102231
1	116745
2	95376
3	75335
4	70826
5	104447
6	248046
7	476708
8	694433
9	728628
10	584603
11	705741
12	676712
13	628078
14	593181
15	521607
16	410292
17	358136
18	263204
19	191429
20	188131
21	161286
22	142094
23	126288

Figure: Hourly distribution

Optimal Enforcement Window Recommendation

The best time to enforce the law is between 8:00 AM and 11:59 AM (hours 8–11), when 2,713,405 tickets were issued, or 32.8% of all violations.

This window has the most violations because it is when morning alternate-side parking enforcement, commercial loading zone restrictions, and peak meter violations all happen at the same time. Hour 9 (728,628 tickets, rank 1), Hour 11 (705,741, rank 2), Hour 8 (694,433, rank 3), and Hour 10 (584,603, rank 5) all have the most tickets in this window. Using the most enforcement officers from 8:00 AM to 12:00 PM would catch almost a third of all daily violations in just 16.7% of the day's hours.

Start_Hour	End_Hour	Total_Tickets
8	11	2713405

Figure: Peak hour

Task 1.3: Geographic Hotspot Identification (Top-K Pattern)

As the lab instructions say, this task uses the Top-K MapReduce pattern. It does this by using local top-K in the mappers and global top-K aggregation in the reducer. The mapper (`mapper_topk.py`) goes through each cleaned CSV row and gets the `street_code1` (column index 9) and `street_name` (column index 24). It makes a composite location key by putting these two fields together with a pipe separator (`street_code|street_name`). The mapper doesn't emit every record; instead, it keeps track of counts in a Python dictionary (`defaultdict`) that is stored in memory. It sorts the dictionary by count in descending order at the end of its input, takes only the top 20 entries, and sends those out as key-value pairs. This cuts down on the amount of data sent to the shuffle phase by a lot.

The reducer (`reducer_topk.py`) gets the top 20 candidates from each mapper. A location could show up in the top 20 lists of more than one mapper with partial counts. The reducer uses a `defaultdict` to combine these partial counts. It sorts the combined counts and picks the global top 20 after processing all the input. It breaks the composite key back down into `street_code` and `street_name` for each entry, figures out the estimated revenue by multiplying the ticket count by an assumed average fine of \$65, and then shows the ranked results.

Mapper/Reducer I/O

Component	Format
Mapper Input	Cleaned CSV row (reads <code>street_code1</code> at index 9, <code>street_name</code> at index 24)
Mapper Output	<code>street_code street_name \t count</code> (local top-20 only)
Reducer Input	Sorted <code>location_key \t count</code> pairs from all mappers
Reducer Output	<code>Rank \t Street_Code \t Street_Name \t Tickets \t Est_Revenue</code>

Top 20 Hotspots

Rank	Street_Code	Street_Name	Tickets	Est_Rever
1	13610	Broadway 64462	4190030	
2	0	WB N CONDUIT AVE @ 8	62968	4092920
3	10210	3rd Ave 51163 3325595		
4	0	WB JAMAICA AVE @ PAR	47814	3107910
5	25390	Madison Ave 40084	2605460	
6	0	EB JAMAICA AVE @ MER	39811	2587715
7	10110	2nd Ave 32985 2144025		
8	24890	Lexington Ave 31746	2063490	
9	10410	5th Ave 29763 1934595		
10	10010	1st Ave 28944 1881360		
11	0	NB CROSS BAY BLVD @	28483	1851395
12	0	SB HAMILTON AVE @ BU	26567	1726855
13	0	SB FRANCIS LEWIS BLV	25486	1656590
14	0	SB MAIN ST @ 37TH AV	24953	1621945
15	10510	6th Ave 24062 1564030		
16	59990	Queens Blvd 23927	1555255	
17	0	NB FRANCIS LEWIS BLV	23884	1552460
18	11710	Amsterdam Ave 21763	1414595	
19	0	WB BRUCKNER BLVD @ R	19823	1288495
20	0	WB ASTORIA BLVD N @	19743	1283295

Figure: Top 20 Hotspots

Enforcement Resource Allocation Recommendation

We suggest sending 40% of enforcement officers to the top 10 places on the list above. These places together have about 429,675 tickets (5.2% of all violations), which brings in about \$27.9 million. With 64,462 tickets a year, Broadway alone needs a dedicated enforcement team. Camera enforcement intersections (street_code=0 entries like WB N CONDUIT AVE and WB JAMAICA AVE) are automated systems that need to be kept up and grown. During the busy hours of 8 AM to 12 PM, traditional enforcement should focus on the main streets (3rd, Madison, 2nd, Lexington, 5th, and 1st).

Task 1.4: Vehicle Risk Profiling (Custom Writable)

This task uses a two-job pipeline and a composite key as the custom writable in Python Streaming. The profile mapper (mapper_profile.py) reads each cleaned CSV row and gets the plate_id (index 1), registration_state (index 2), plate_type (index 3), and vehicle_color (index 33). It makes a composite vehicle key by putting plate_id, state, and plate_type together with pipe separators (for example, "JDL9152|NY|PAS"). It then sends this key with the vehicle color as the value. The

default partitioning for Hadoop Streaming puts all records with the same vehicle key in the same reducer.

The profile reducer (reducer_profile.py) uses the streaming pattern to process all color values for each vehicle key. It keeps track of the current vehicle key, the total count, and a color frequency dictionary. If the key of an incoming record matches the current vehicle, it adds one to the count and changes the color frequency. When a new key comes in, it clears the profile of the previous vehicle. It chooses the dominant color (the most common one, with alphabetical tie-breaking) and sends out the vehicle key, dominant color, and total ticket count.

The risk mapper (mapper_risk.py) reads the vehicle profiles and sends all records back out with the constant key "ALL" to make sure they all go to the same reducer. In one pass through the data, the risk reducer (reducer_risk.py) does three analyses. First, it adds up the total number of tickets and vehicles for each color to find the color risk scores. This is done by dividing the average number of tickets per vehicle for each color by the overall average. Second, it puts vehicles into groups based on how many times they've broken the law (5–10, 11–20, and 21+ violations). Third, it keeps track of the car with the most tickets as the worst offender.

Mapper/Reducer I/O

Job 1 (Vehicle Profiles):

Component	Format
Mapper Input	Cleaned CSV row (reads plate_id, registration_state, plate_type, vehicle_color)
Mapper Output	vehicle_key \t color
Reducer Input	Sorted vehicle_key \t color pairs (grouped by vehicle)
Reducer Output	vehicle_key \t dominant_color \t ticket_count

Job 2 (Risk Analysis):

Component	Format
Mapper Input	vehicle_key \t dominant_color \t ticket_count
Mapper Output	ALL \t vehicle_key \t color \t count (single key for global analysis)
Reducer Input	All vehicle profiles in one reducer
Reducer Output	Color risk scores, repeat offender buckets, top offender

Color Risk Analysis

Color	Tickets	% of Total	Risk Score
WHITE	2,027,693	27.33%	1.16
GRAY	2,025,216	27.29%	0.92
BLACK	1,778,612	23.97%	0.98
BLUE	634,552	8.55%	0.92
RED	386,186	5.20%	0.91
BROWN	149,262	2.01%	2.18
SILVER	102,462	1.38%	0.76
YELLOW	70,931	0.96%	1.15

Repeat Offender Patterns

Violation Range	Vehicle Count
5-10 violations	292,617
11-20 violations	69,201
21+ violations (chronic offenders)	23,949

Top Offender: BLANKPLATE|99|999 (BLACK) with 17,379 violations. This represents records where the plate could not be read by camera enforcement systems.

Task 1.5: Revenue Analysis by Violation Type (Map-Side Join)

This task uses Hadoop's Distributed Cache to carry out the Map-Side Join pattern. The revenue mapper (mapper_revenue.py) first loads the fine_lookup.csv file into a dictionary that is kept in memory. It does this before it starts processing any input records. The Hadoop Streaming command's -files flag sends this file to all mapper instances. The lookup maps violation codes to their labels and the amount of the fine (for example, code 14 maps to "No Standing" with a \$115 fine). The mapper gets the violation_code (index 5) from each cleaned CSV row and looks it up in the dictionary. If it finds it, it uses the mapped label and fine. If not, it gives it a default label of "OTHER" and a fine of \$50. It makes a composite key out of the violation code, label, and fine,

and then sends it out with a ticket count of 1 and the fine amount as revenue. There is no need for a join on the reducer side; this join happens entirely in the map phase.

Using the standard streaming aggregation pattern, the revenue reducer (reducer_revenue.py) adds up the ticket counts and revenue totals for each composite violation key. The share mapper (mapper_share.py) then sends all records with the key "ALL" to a single reducer. The share reducer (reducer_share.py) gathers all types of violations, sorts them by revenue from highest to lowest, gives them ranks, figures out what percentage of total revenue each type makes, and finds the type of violation that makes the most money.

Mapper/Reducer I/O

Job 1 (Revenue Aggregation):

Component	Format
Mapper Input	Cleaned CSV row + fine_lookup.csv (via Distributed Cache)
Mapper Output	violation_code label fine \t 1 \t revenue
Reducer Input	Sorted violation_key \t ticket_count \t revenue pairs
Reducer Output	code \t label \t fine \t total_tickets \t total_revenue

Job 2 (Revenue Ranking):

Component	Format
Mapper Input	code \t label \t fine \t tickets \t revenue
Mapper Output	ALL \t code \t label \t fine \t tickets \t revenue
Reducer Input	All violation types in one reducer
Reducer Output	Rank \t Code \t Label \t Fine \t Tickets \t Revenue \t Pct_Revenue

Revenue Analysis Results

Code	Violation Type	Fine	Tickets	Revenue	% Rev
36	OTHER (Speed Camera)	\$50	2,717,237	\$135,861,850	28.99%

14	No Standing	\$115	406,679	\$46,768,085	9.98%
21	Street Cleaning	\$45	941,475	\$42,366,375	9.04%
40	Fire Hydrant	\$115	306,600	\$35,259,000	7.52%
38	Meter Expired	\$35	592,069	\$20,722,415	4.42%
20	No Parking	\$65	296,651	\$19,282,315	4.11%
46	Double Parking	\$115	166,982	\$19,202,930	4.10%
71	Improper Display	\$65	264,268	\$17,177,420	3.67%

Total Estimated Revenue: \$468,629,155

Most Profitable Violation: Code 36 (School Zone Speed Camera) at \$135.9M (29.0% of total revenue from 32.8% of tickets). This code generates the highest revenue due to sheer volume despite having a moderate \$50 fine. No Standing violations (Code 14) are the most profitable per-ticket at \$115, generating \$46.8M from only 4.9% of tickets.

Part 2: E-Commerce User Journey Analytics

Task 2.1: User Session Reconstruction (Secondary Sort)

This task uses the Secondary Sort pattern to reconstruct user sessions. The mapper auto-detects the input delimiter (comma or tab), strips Windows line endings, and normalizes timestamps to a sortable YYYY-MM-DD HH:MM:SS format. It emits user_id and timestamp as a composite key. Hadoop is configured with three settings: stream.num.map.output.key.fields=2 defines the composite key, the partitioner groups by user_id only (-k1,1), and the comparator sorts by user_id then timestamp (-k1,1 -k2,2). The reducer processes each user's chronologically sorted events, maintaining session state. It splits sessions when the gap exceeds 30 minutes or events cross midnight. Each session output includes duration, event count, conversion flag, revenue, and view/cart/purchase indicators. Summary metrics are printed at the end.

Mapper/Reducer I/O

Component	Format
Mapper Input	Raw clickstream CSV/TSV (10 columns: event_id, user_id, timestamp, ...)

Mapper Output	user_id \t normalized_timestamp \t event_id \t event_type \t price \t device \t source
Reducer Input	Events sorted by user_id then timestamp (via secondary sort)
Reducer Output	session_id, user_id, start_time, end_time, duration, events, converted, revenue, device, source, has_view, has_cart, has_purchase

Required Metrics

Metric	Value
Total sessions reconstructed	97,038
Average session duration	23.8 minutes
Conversion rate (% sessions with purchases)	34.3%
Average events per session	10.3
Average revenue per converted session	\$606.13

Golden Session Pattern

Our analysis shows that the golden session pattern with the highest chance of conversion has these traits: Duration of 20–40 minutes (long enough for browsing but showing focused intent), 10–15 events (view-addtocart-purchase funnel completion), desktop device (higher conversion rate than mobile/tablet because checkout is easier), and direct or search traffic source (showing higher purchase intent).

Suggested ways to get more people to follow this pattern: (1) Make sure that mobile checkout works as well as desktop checkout. (2) Use exit-intent popups for sessions that are about to reach 15 minutes without adding to the cart. (3) Give personalized product suggestions after 5 or more page views to speed up the funnel. (4) Use retargeting campaigns for people who put things in their cart but didn't buy them right away.

Task 2.2: Customer Segmentation using K-Means Clustering

This task uses iterative K-means in three steps. The feature mapper gets end_time, converted flag, and revenue from the session data. The feature reducer calculates normalized RFM features for each user. $R = \text{days since last activity} / 90$, $F = \text{sessions} / 50$, $M = \text{revenue} / 5000$, and all of these values are capped at 1.0. The reference date is calculated automatically from the data. The K-means mapper gets centroids from Distributed Cache, uses Euclidean distance to assign each user to the closest centroid, and a combiner does partial aggregation to cut down on network traffic.

The reducer finds new centroids by taking the average of the users it has assigned. Every time the loop runs, it checks for convergence by comparing the old and new centroids. The loop ends when the maximum movement drops below 0.001 or after 15 iterations. The final summary reducer uses rank-based scoring ($R + F + M$) to give segment labels and groups clusters into Champions, Potential Loyalists, At Risk, and New/Casual.

Mapper/Reducer I/O

Feature Extraction Job:

Component	Format
Mapper Input	Session CSV (from Task 2.1 output)
Mapper Output	user_id \t end_time \t converted \t revenue
Reducer Input	Sorted user events
Reducer Output	user_id, R, F, M (normalized RFM features)

K-Means Iteration Job (with Combiner):

Component	Format
Mapper Input	user_id, R, F, M + centroids.txt (via Distributed Cache)
Mapper Output	cluster_id \t 1 \t R \t F \t M
Reducer Input	Partially aggregated sums per cluster (via combiner)
Reducer Output	cluster_id, new_R, new_F, new_M (updated centroid)

Convergence Report

The algorithm converged within the maximum iteration limit. The final maximum centroid movement fell below the 0.001 threshold, indicating stable cluster assignments. The convergence check compares Euclidean distance between old and new centroids after each iteration, and the iterative loop terminates when all centroids move less than 0.001.

Cluster Analysis and Customer Segments

Cluster	Users	%	R	F	M	Segment
0	3,559	18.3%	0.599	0.393	0.971	Champions

2	3,087	15.9%	0.643	0.273	0.504	Potential Loyalists
1	4,430	22.8%	0.659	0.255	0.182	At Risk
3	8,372	43.0%	0.678	0.062	0.008	New/Casual

Segment Profiles and Marketing Strategies

Cluster 0 - "Champions" (18.3% of customers): Centroid: R=0.599, F=0.393, M=0.971. Profile: Most frequent visitors with the highest spending. These customers have the strongest engagement and monetary contribution. Strategy: VIP loyalty programs, exclusive early access to sales, personalized premium recommendations. Expected impact: Retain these high-value customers to protect ~40% of total revenue.

Cluster 2 - "Potential Loyalists" (15.9% of customers): Centroid: R=0.643, F=0.273, M=0.504. Profile: Growing engagement with moderate spending. These customers show promise but haven't reached champion status. Strategy: Personalized product recommendations, upgrade incentives, loyalty program enrollment, targeted cross-sell campaigns. Expected impact: Converting 20% of this segment to Champions could increase revenue by ~15%.

Cluster 1 - "At Risk" (22.8% of customers): Centroid: R=0.659, F=0.255, M=0.182. Profile: Moderate frequency but declining monetary value. These customers may be losing interest. Strategy: Win-back campaigns with special discount offers, re-engagement email sequences, feedback surveys to understand declining spend. Expected impact: Preventing churn of this segment preserves ~10% of revenue.

Cluster 3 - "New/Casual" (43.0% of customers): Centroid: R=0.678, F=0.062, M=0.008. Profile: Largest segment with minimal engagement and near-zero spending. Many are new users or one-time visitors. Strategy: Welcome email series, educational content about product offerings, first-purchase incentives, simplified onboarding flow. Expected impact: Converting even 5% of this large segment to active buyers would significantly expand the customer base.

Video Demonstration

<https://drive.google.com/file/d/13Au8-GhB1IaExhMk2wEmY3GAGKf3Q8E5/view?usp=sharing>